

# Fraud Detection Banking Dashboard

Project Proposal

Extended with Banking Module | July 2026

Banking & Fraud Intelligence Platform

# Table of Contents

1. Introduction
2. Problem Statement
3. Project Objectives
4. Project Scope
5. System Architecture
6. Project Flow
7. Technologies and Tools
8. Functional Requirements
9. Non-Functional Requirements
10. Expected Outcomes
11. Conclusion

# 1 Introduction

The rapid growth of online banking and digital payment systems has increased both convenience and fraud risks. This project proposes a web-based Fraud Detection Banking Dashboard that combines rule-based analysis, machine learning, and real-time banking capabilities to detect suspicious transactions, generate fraud alerts, support fraud investigations, and provide end-users with secure banking services including account management and money transfers.

The platform is built with a modern technology stack — Next.js 16 on the frontend, Spring Boot 4 on the backend, Supabase for database and authentication, and a Python sidecar for machine learning fraud prediction. The interface is role-aware: fraud analysts and administrators see the fraud intelligence console, while regular users see a personal banking dashboard.

## 2 Problem Statement

Manual monitoring of thousands of daily banking transactions is inefficient and delays the detection of fraudulent activities. Banks require an automated system to identify suspicious transactions, reduce financial losses, and improve investigation efficiency. Banking customers also need a modern, secure interface to manage their accounts and perform transfers, all while benefiting from the same fraud detection infrastructure that protects their transactions.

## 3 Project Objectives

### 3.1 General Objective

The main objective is to develop a web-based Fraud Detection Banking Dashboard that can monitor banking transactions, identify suspicious activities, support fraud case investigation, and provide secure banking services to end-users.

### 3.2 Specific Objectives

- Develop a secure login system with role-based authentication for banking staff, fraud analysts, and end-users.
- Collect and store customer, account, and transaction information.
- Display banking transactions through an interactive dashboard with real-time updates.
- Analyze transactions using predefined fraud detection rules.
- Connect the Java backend with a Python machine learning service for fraud prediction.
- Calculate a fraud probability and risk level for each transaction.
- Classify transactions into low, medium, high, or critical risk levels.
- Automatically create fraud alerts for suspicious transactions.
- Allow fraud analysts to review and investigate fraud alerts.
- Provide fraud case management: create cases, add notes, assign analysts, update status.
- Record important user activities through audit logs.
- Generate charts and reports on transactions, fraud alerts, cases, and financial losses.
- Provide a banking module for end-users to manage accounts, send/receive money, and view transfer history.
- Implement atomic money transfers using database-level transactions and RPC functions.

## 4 Project Scope

The project focuses on developing a prototype banking fraud monitoring and case management system with a built-in banking module for end-users.

### 4.1 User Authentication and Authorization

Users log in using their email address and password. Access to system functions is controlled by role. Roles are persisted in the database and fetched on each session, with a fallback to signup metadata.

The proposed roles are:

- Administrator — Full system access: manage users, configure fraud rules, view all data.
- Investigator — Investigate suspicious transactions, manage fraud cases, view alerts and analytics.
- Analyst — View transactions, alerts, cases, and generate reports.
- User — Access personal banking dashboard, view accounts, send and receive money, view transfer history.

### 4.2 Banking Module

The banking module provides end-users with a full-featured banking interface within the same application:

- Account Management — Each user is automatically assigned a Main Checking account (\$10,000 starting balance) on signup, plus optional Savings, Travel Card, and Investment accounts.
- Balance Overview — Real-time display of all account balances and total portfolio value.
- Send Money — Users transfer funds by searching via account number or email. A two-step flow verifies the recipient (name, account number, email) before confirming.
- Receive Money — Account details displayed for sharing: account number, name, and bank information.
- Transfer History — Complete list of transfers with status indicators (completed, pending, failed).
- Atomic Transfers — All money movements use a PostgreSQL RPC function with row-level locking for atomic debit/credit.

### 4.3 Dashboard

The dashboard provides two views based on user role:

- Fraud Intelligence Dashboard (admin, investigator, analyst) — Summary of fraud metrics: total transactions, suspicious transactions, open alerts, confirmed cases, risk distribution, trends.
- Banking Overview (user) — Personal financial summary: total balance, account details, recent transfers, quick actions.

### 4.4 Transaction Management

Authorized users can:

- View transactions with real-time updates via Supabase Realtime subscriptions.
- Search and filter by date, amount, status, transaction type, and risk level.
- View detailed transaction info: customer, account, fraud score, rule triggers.

### 4.5 Rule-Based Fraud Detection

The system analyzes transactions using predefined rules stored in the database:

- Transactions exceeding \$10,000 (High-Value Threshold).
- More than 5 transactions in 10 minutes (Velocity Spike).
- Transactions from unusual geographic locations (Geo Anomaly).
- Transactions from unrecognized devices (Device Mismatch).
- High-value transactions during 12AM-5AM (Night Trading).
- Rapid ATM cash-out patterns (Rapid Cash-Out).
- Accounts less than 30 days old with large transactions (New Account Risk).
- Cross-border transactions to high-risk regions (International Alert).

Each matched rule contributes to the risk score. Administrators configure rules through the Rules Engine interface.

## 4.6 Machine Learning Fraud Prediction

The Spring Boot backend sends selected transaction features to a Python ML service (Flask/FastAPI). The service uses a pre-trained Hugging Face transformer model fine-tuned for fraud detection, or alternatively a custom Scikit-learn model trained on historical transaction data. The service returns fraud probability, predicted status, risk level, and contributing factors. The backend combines the rule-based score and ML prediction for the final risk classification.

## 4.7 Fraud Alert Management

When a transaction is flagged, the system automatically creates a fraud alert. Analysts can:

- View alerts sorted by severity and timestamp.
- Filter by risk level and read/unread status.
- Assign alerts to analysts for investigation.
- Review full transaction details with rule triggers and ML predictions.
- Update alert status: New, Under Review, Escalated, Resolved, Dismissed.
- Escalate critical alerts and create fraud cases from confirmed activity.

## 4.8 Fraud Case Management

Analysts can create and manage investigation cases:

- Link cases to fraud alerts and transactions.
- Assign analysts and track progress.
- Add notes with type classification: investigation, resolution, update, comment.
- Track status: Open, In Progress, Escalated, Confirmed Fraud, False Positive, Closed.

## 4.9 Reporting and Analytics

The system provides visual reports and charts:

- Daily and monthly fraud trends.
- Cases by status and severity.
- Transactions by risk level and type.
- Geographic fraud distribution.
- Confirmed fraud vs false positives.
- Financial loss tracking and prevention metrics.
- Analyst investigation activity and resolution performance.

## 4.10 Audit Logging

The system records important activities:

- User login/logout events.
- Alert review and status changes.
- Case creation, updates, and resolution.
- Analyst assignment changes.
- User management operations.
- All logs are timestamped and attributed to the acting user.

## 4.11 Database Schema

The system uses the following Supabase PostgreSQL tables:

- `user_profiles` — Extends `auth.users` with role, email, full name. Created via trigger on signup.
- `transactions` — Banking transactions with risk scores, ML predictions, rule triggers, device/geo data.
- `fraud_cases` — Investigation cases linked to suspicious transactions.
- `case_notes` — Investigation notes attached to fraud cases.
- `alerts` — Fraud alerts with severity, read status, and assignment.
- `fraud_rules` — Configurable detection rules with conditions, actions, and hit counters.
- `accounts` — User-owned bank accounts with balance, currency, and unique account number. Default \$10,000 Main Checking created on signup.
- `transfers` — Money transfer records with sender/receiver, amount, note, and status.

Key PostgreSQL functions:

- `transfer_money(sender, receiver, amount, note)` — Atomic debit/credit using `SELECT FOR UPDATE`. Validates balance, debits sender, credits receiver, inserts record in one transaction.
- `lookup_recipient(query)` — Searches accounts by number or email. `SECURITY DEFINER` to bypass RLS for cross-user lookups.
- `handle_new_user()` — Trigger on `auth.users` insert. Creates profile + default Main Checking account (\$10,000).

## 4.12 Project Limitations

This is an academic prototype, not connected to a real banking system. Transaction data may be entered manually, system-generated, or imported from sample datasets. ML accuracy depends on training data quality. The system assists analysts but does not make final legal or financial decisions. Features like automatic account freezing, law enforcement communication, ATM integration, and payment provider connections are not included.



# 5 System Architecture

The system uses a multi-layer architecture with four main components.

## 5.1 Frontend Layer

Built with Next.js 16, TypeScript, and Tailwind CSS 4. The interface is role-aware: users see a banking dashboard, while analysts/investigators/admins see the fraud intelligence console. The frontend communicates with Supabase directly for data queries and authentication, and with the Spring Boot backend for business logic.

## 5.2 Backend Layer

Developed with Spring Boot 4. Responsibilities:

- Process frontend requests and validate transaction data.
- Apply business rules and orchestrate fraud rule evaluation.
- Manage users, alerts, and investigation cases.
- Communicate with Supabase for data storage.
- Call the Python ML service for fraud predictions.
- Expose REST API endpoints.

## 5.3 Database and Authentication Layer

Supabase provides PostgreSQL and authentication. Schemas use idempotent migrations. Row-Level Security controls data access per role.

Authentication flow:

- Signup: `handle_new_user` trigger creates profile + Main Checking account (\$10,000).
- Login: role fetched from `user_profiles` table, falls back to `user_metadata`.
- Session: role controls sidebar visibility, page rendering, and data access.

## 5.4 Machine Learning Layer

A Python ML sidecar (Flask/FastAPI) serves predictions using Hugging Face transformer models (e.g. fine-tuned BERT or tabular classifiers) or custom Scikit-learn models. It receives transaction features from Spring Boot via HTTP and returns fraud probability, risk level, and contributing factors.

## 5.5 Architecture Overview

Frontend (Next.js 16) !" Spring Boot Backend API !" Supabase (PostgreSQL + Auth)

Spring Boot Backend API !" Python ML Service (Fraud Prediction)

## 6 Project Flow

### 6.1 Fraud Detection Flow

- User Login !' Authentication !' Role Verification
- Open Dashboard (fraud intelligence or banking view based on role)
- Receive or Enter Transaction Data
- Validate and Save Transaction to Database
- Apply Predefined Fraud Rules (scored per rule)
- Send Transaction Features to ML Service
- Receive Fraud Probability and Risk Level
- Combine Rule Score + ML Prediction !' Final Risk Classification
- If Suspicious !' Create Fraud Alert Automatically
- Fraud Analyst Reviews Alert !' Opens Investigation Case !' Resolution

### 6.2 Banking Transfer Flow

- User Opens Banking Section (TransferView)
- Selects Send Money Tab
- Enters Recipient (account number or email)
- System Looks Up Recipient via lookup\_recipient RPC !' Shows Name, Account, Email
- User Verifies and Confirms !' Enters Amount + Note
- System Calls transfer\_money RPC (atomic transaction)
- SELECT FOR UPDATE locks sender row !' Validates Balance
- Debits Sender, Credits Receiver in Same Transaction
- Inserts Transfer Record with Status = completed
- Both Users See Updated Balances and History

# 7 Technologies and Tools

## Frontend

- Next.js 16, TypeScript, Tailwind CSS 4, Recharts, Framer Motion, Lucide React

## Backend

- Java, Spring Boot 4, Spring Web, Spring Security, Spring Data JPA, REST API, Maven

## Database & Authentication

- Supabase, PostgreSQL, Supabase Auth (JWT), Row-Level Security, PL/pgSQL RPC Functions, Realtime Subscriptions

## Machine Learning

- Python, Flask/FastAPI, Pandas, NumPy, Scikit-learn, Hugging Face Transformers, Joblib

## Development Tools

- VS Code, IntelliJ IDEA, Postman, Git, GitHub, Supabase Dashboard, Draw.io

## Testing

- JUnit, Mockito, Postman, Browser DevTools, ESLint, TypeScript

## 8 Functional Requirements

- Authenticate users via email/password with Supabase Auth.
- Authorize users according to assigned roles (admin, investigator, analyst, user).
- Create, read, update, and search banking transaction records.
- Analyze transactions using configurable fraud detection rules with severity scoring.
- Send transaction features to the Python ML service and receive fraud predictions.
- Calculate combined transaction risk levels from rule + ML scores.
- Automatically create alerts for suspicious transactions.
- Allow analysts to investigate alerts, update status, and create cases.
- Store investigation notes with type classification and author attribution.
- Display fraud statistics and interactive charts with real-time updates.
- Generate fraud reports with filtering and export capabilities.
- Record important user actions in an immutable audit log.
- Allow administrators to manage users, roles, and fraud rule configuration.
- Provide a banking module with account management and money transfers.
- Execute atomic money transfers with balance validation and row-level locking.
- Support recipient lookup by account number or email.

## 9 Non-Functional Requirements

- Security — Role-based authentication, Row-Level Security in PostgreSQL, JWT-based API calls, SECURITY DEFINER RPCs for sensitive operations.
- Performance — Fast dashboard response with lazy loading, paginated data, Supabase Realtime for live transaction updates.
- Reliability — Input validation, comprehensive audit logging, atomic database transactions for money transfers.
- Usability — Clean, intuitive interface with role-aware views and responsive design.
- Maintainability — Modular React components, idempotent database migrations, TypeScript throughout, clean separation of concerns.
- Scalability — Supabase horizontal scaling, Next.js edge rendering, stateless Spring Boot backend, independent ML service.

## 10 Expected Outcomes

- A functional web-based banking fraud detection and management dashboard.
- A secure authentication and role-based access control system supporting four roles.
- A centralized interface for monitoring banking transactions with real-time updates.
- Automated rule-based fraud detection with configurable rules and severity scoring.
- Integration between the Spring Boot backend and Python ML service for fraud prediction.
- Fraud probability and risk-level calculation combining rule-based and ML scores.
- Automatic generation of fraud alerts with severity classification and analyst assignment.
- A fraud investigation and case management module with full audit trail and note tracking.
- Visual reports and charts showing fraud trends, case status, and system performance.
- Immutable audit logs recording all important user activities.
- A functional banking module for end-users: accounts, send/receive money, transfer history.
- Atomic money transfer operations using PostgreSQL RPCs ensuring data integrity.
- Improved speed and consistency in identifying suspicious banking transactions.
- Better support for fraud analysts in investigation and resolution decisions.
- A modular, extensible architecture supporting additional rules, ML models, and banking features.

# 11 Conclusion

The proposed Fraud Detection Banking Dashboard demonstrates how modern web technologies, cloud databases, real-time data synchronization, and machine learning can work together to improve fraud detection while simultaneously providing end-users with a secure, modern banking experience.

The addition of the banking module transforms the application from a pure monitoring tool into a complete banking platform where fraud detection protects every transaction in real time. Users benefit from the same security infrastructure that protects the bank's systems, creating a unified experience that prioritizes both security and usability.

Although developed as an academic prototype, the system provides a strong foundation for future real-world banking applications. The modular architecture, idempotent database design, and separation of concerns allow for incremental enhancement — whether adding new fraud detection rules, training improved ML models, or expanding banking functionality.